ABSTRACT

SlotFinder is a full-stack vehicle parking web application designed to identify free parking slots for a chosen vehicle type and time window, and to let users book them without any possibility of double booking. The system provides separate capabilities for users and an administrator. Users can register and log in, search for free slots by vehicle type (car, bike, EV or truck) and time, view the estimated cost before booking, follow live slot occupancy on a colour-coded slot map, inspect a 24-hour day-view timeline for every slot, book a slot with their vehicle number, and view or cancel their bookings. When every slot is occupied, the system suggests the earliest time a slot becomes free for the same duration. The administrator can add, edit, deactivate and delete parking slots, view and cancel any booking, and monitor booking and revenue statistics.

The frontend is implemented using HTML5, CSS3 and vanilla JavaScript, with the Fetch API used for communication with the server. The backend uses Node.js and Express to expose REST-style API routes. MongoDB stores application records through Mongoose models. JSON Web Tokens (JWT) and bcrypt password hashing support authentication, while express-validator enforces input rules on the server. The core of the project is a time-interval overlap algorithm that compares every slot against existing bookings to identify free slots, combined with a re-check at booking time that prevents double booking. The project demonstrates the integration of client-side interfaces, server-side business logic, database persistence and role-based access in a single web application.

The report follows the supplied Web Technology III IA project contents: introduction, literature survey, methodology and system design, implementation and results, conclusion and future enhancement, and references. The screenshots in this report were taken from the running application, and the test results were recorded by executing the project's automated test suite.

TABLE OF CONTENTS

S.NO	CONTENTS	PG.NO
1	Introduction	1 – 3
2	Literature Survey	4 – 5
3	Methodology and System Design	6 – 11
4	Implementation and Results	12 – 19
5	Conclusion and Future Enhancement	20 – 21
6	References	22

CHAPTER 1: INTRODUCTION

1.1 Background of the Project

Vehicle parking is a daily need in offices, colleges, hospitals, shopping centres and residential complexes. As the number of vehicles grows, drivers often arrive at a parking facility without knowing whether a suitable space is free. They circle the parking area, waste fuel and time, and add to congestion. Parking operators, on the other hand, track spaces and reservations manually or with simple tools that do not show which slot is occupied at a given time.

SlotFinder addresses these needs through one web application. It treats every parking slot as a resource that can be reserved for a specific time window and checks each slot against existing bookings before showing it as free. The platform separates user and administrator tasks. Users search for and book free slots for their vehicle, and the administrator manages slots, prices and bookings. This makes the project a practical demonstration of modern web technology concepts such as client-server communication, REST APIs, database modelling, authentication, validation and algorithm design.

A parking booking system must coordinate several related activities rather than display a list of spaces alone. Slot information has to remain consistent while users search, compare, book and cancel. At the same time, two users may try to book the same slot for overlapping times, and the system must decide fairly which booking succeeds. Different vehicle types also need different slots and prices.

SlotFinder organizes these activities into role-based areas within a single web application. The user focuses on finding a free slot, booking it and managing bookings, while the administrator focuses on maintaining the slot inventory and monitoring bookings. Live occupancy views and a day timeline make the state of the parking facility visible to everyone at a glance.

1.2 Problem Statement

A basic parking booking website may support a booking form but may not check whether a particular slot is actually free for the requested time. Many systems only count the total number of spaces, so two users can be given the same slot for overlapping periods. When the parking is full, users usually receive a "no slots available" message with no indication of when a slot will become free. The problem is to develop an integrated system that identifies

truly free slots for a given vehicle type and time window, prevents double booking, and provides role-based access, persistent records and administrative monitoring.

When these activities are handled manually or through separate tools, an operator may need to check registers, spreadsheets and phone messages to know which slot is free. This makes it difficult to trace a slot from booking to completion or to resolve conflicts between two reservations. A parking system also needs controls on vehicle numbers, time windows and slot details so that the records are not dependent on unchecked user input.

The project therefore addresses the need for a connected workflow in which user actions are sent to backend APIs, availability is computed from stored bookings, and the resulting records are stored in a database. The goal is to demonstrate a practical web technology solution, not to claim that every operational issue of a commercial parking facility is eliminated.

1.3 Objectives

- To develop a responsive web interface for searching and booking parking slots.
- To provide user registration, login and JWT-based authenticated access to protected features.
- To identify free slots for a selected vehicle type and time window using a time-overlap algorithm.
- To show the estimated cost of each free slot before booking.
- To prevent double booking of a slot and to stop the same vehicle from holding two overlapping bookings.
- To suggest the next available time when every slot of the required type is occupied.
- To display live slot occupancy and a 24-hour day-view timeline for every slot.
- To provide administrator workflows for adding, editing, deactivating and deleting slots and for monitoring bookings and revenue.
- To store application data in MongoDB and expose backend operations through Express API routes.
- To validate every input both in the browser and on the server.

1.4 Scope of the Project

The scope includes the public home page, login and registration pages, the user pages for finding slots, the day view and booking management, the administrator panel, the backend

REST APIs and MongoDB persistence. The implemented project structure includes models for users, slots and bookings. Four vehicle types are supported: car, bike, EV and truck. The booking amount is calculated as the slot's price per hour multiplied by the booked duration. Online payment, physical sensors and deployment to a public server are outside the claims made by this report.

The project scope includes account access, free-slot search, live occupancy, the day timeline, booking and cancellation, next-free suggestions, administrative slot management and booking statistics. The implementation also includes backend models, middleware and API routes that support the main modules, together with an automated API test suite.

The scope does not include a payment gateway or integration with gate barriers or parking sensors. Occupancy is derived from bookings recorded in the system rather than from detecting vehicles physically. For an academic demonstration, test accounts and sample slots created by the seed script are used, and real passwords or personal details should not appear in screenshots.

CHAPTER 2: LITERATURE SURVEY

2.1 Review of Existing Websites / Web Applications

Existing parking and vehicle booking services provide useful reference patterns for availability display, reservation and payment. Airport and mall parking websites generally allow a driver to pre-book a space for a date and time. City parking applications show nearby parking locations on a map with indicative availability. Smart parking solutions add sensors or cameras at each space to detect whether it is physically occupied.

The project uses these common patterns as design references rather than claiming that it reproduces the complete internal systems of any commercial service. The comparison below is conceptual and based on publicly visible feature categories; it is not the result of a controlled performance evaluation.

Commercial parking platforms commonly provide location search, date and time selection, pricing, booking confirmation and booking history. Many of them present availability as a total count of spaces for a car park rather than as individual slots. Sensor-based systems show real-time occupancy but require hardware installed at every space. These patterns are useful references for understanding how users expect a parking booking system to behave.

This report uses these general patterns as a functional comparison. It does not claim access to the private implementation details or internal databases of commercial platforms. The comparison is based on visible user-facing concepts and on the features present in the SlotFinder source project.

2.2 Summary of Existing Approaches

- Manual counter approach: an attendant allots a space when the vehicle arrives. It needs no software but gives drivers no information in advance and cannot prevent conflicts between advance reservations.
- Count-based online booking approach: the website stores the total number of spaces and reduces the count for each booking. It is simple but does not track which slot is free for which time window, so overlapping bookings are possible.
- Sensor-based smart parking approach: sensors detect occupied spaces in real time. It is accurate for the present moment but needs hardware for every space and does not support reservations for a future time window by itself.

- SlotFinder approach: tracks individual slots per vehicle type, checks every slot against existing bookings for the exact requested time window, prevents double booking, suggests the next free time and shows live occupancy and a day timeline, without requiring sensor hardware.

2.3 Research Gap or Limitations of Existing Systems

From the perspective of this student project, the main design gap is the need to identify free slots per time window rather than per car park. A simple booking form may not check overlapping reservations, while a sensor-based system may show who is parked now but not who will arrive in two hours. SlotFinder attempts to integrate time-based availability, booking, conflict prevention and visual occupancy in one workflow. This is a functional design objective, not a claim that no existing commercial platform offers similar capabilities.

The current project also has limitations. Occupancy is based on bookings rather than physical detection, so a vehicle that parks without booking is not recorded. Payment is not integrated, and real deployment requires hardened configuration, secure secret management, backups, monitoring and broader security and load testing.

From the project perspective, the design challenge is to decide reliably whether two time intervals overlap and to make that decision at the moment a booking is created. A booking can be upcoming, active, completed or cancelled, and only active and upcoming bookings should block a slot. The application must also separate the actions available to users and administrators.

SlotFinder demonstrates a student-project approach to these requirements, but its reliability in a public deployment would depend on security review, concurrency handling under heavy load, accessibility checks and load testing. These are important limitations to identify rather than assume they have already been solved.

CHAPTER 3: METHODOLOGY AND SYSTEM DESIGN

3.1 Proposed Methodology

The application follows an iterative development process. First, the user roles and workflows are identified. Next, the frontend pages and navigation are organized by user and administrator tasks. Backend routes define the operations available to the frontend, route handlers implement the application logic, and Mongoose models represent persistent records. Authentication and role checks protect restricted functions. Finally, the application is checked using representative workflows such as registration, free-slot search, booking, double-booking attempts, cancellation and administrative slot management.

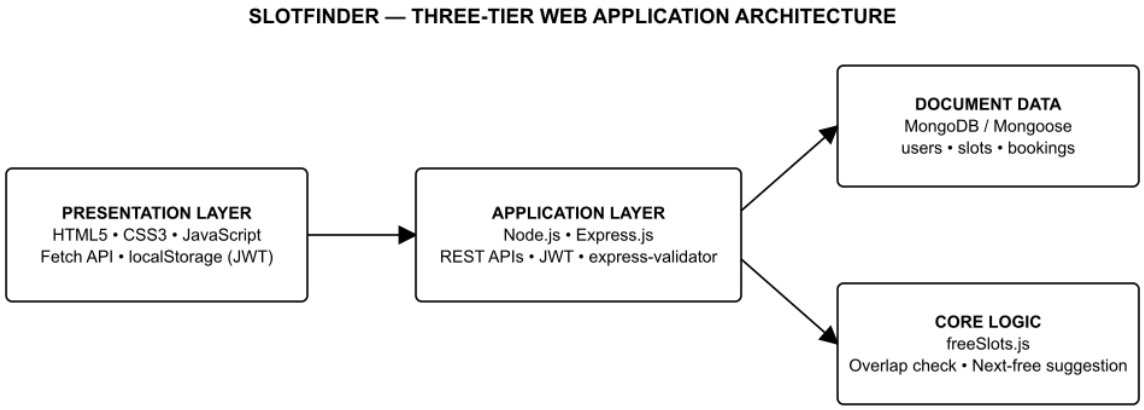
1. Requirement analysis: identify roles, pages, vehicle types, slot states and booking workflows.
2. Interface design: build HTML pages, shared styles, navigation, live validation and status feedback.
3. Algorithm design: define the time-overlap rule, the free-slot search and the next-free suggestion.
4. API development: implement Express routes and middleware for authentication, slots and bookings.
5. Data modeling: define MongoDB collections through Mongoose schemas and relationships using document references.
6. Validation: run automated API tests and manual checks, record outcomes and correct errors before submission.

The development methodology begins with identifying the main actors and their tasks. These tasks are then mapped to pages and backend endpoints. Data models are used to represent the information needed by each module, and the frontend communicates with the backend rather than accessing the database directly. This division creates a clearer boundary between presentation, business rules and persistence.

Integration is important because a page may appear correctly while an API call or database operation still fails. For that reason, each workflow is checked across the complete path: user action in the browser, request sent to the backend, validation and authorization, free-slot computation, database operation, and response shown to the user.

3.2 System Architecture

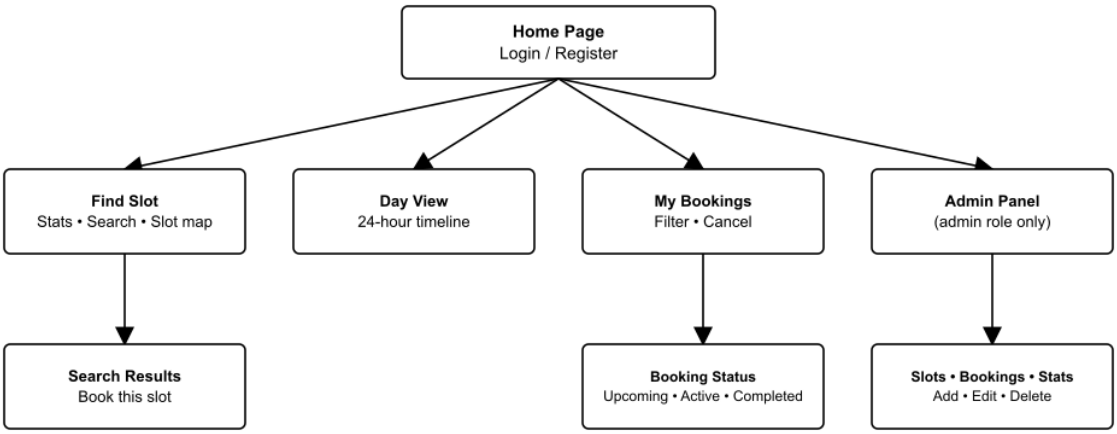
SlotFinder follows a three-tier logical architecture based on the Model-View-Controller (MVC) pattern. The presentation tier consists of the HTML, CSS and JavaScript pages running in the browser. The application tier is the Node.js/Express server, which exposes API routes, validates requests, applies authentication and authorization middleware and runs the free-slot identification logic. The data tier is MongoDB, accessed through Mongoose. The Mongoose models act as the Model, the browser pages as the View, and the Express route handlers as the Controller.



This is a logical architecture diagram based on the source code and package configuration. In the development environment all components run on one machine, with MongoDB available either locally or on MongoDB Atlas.

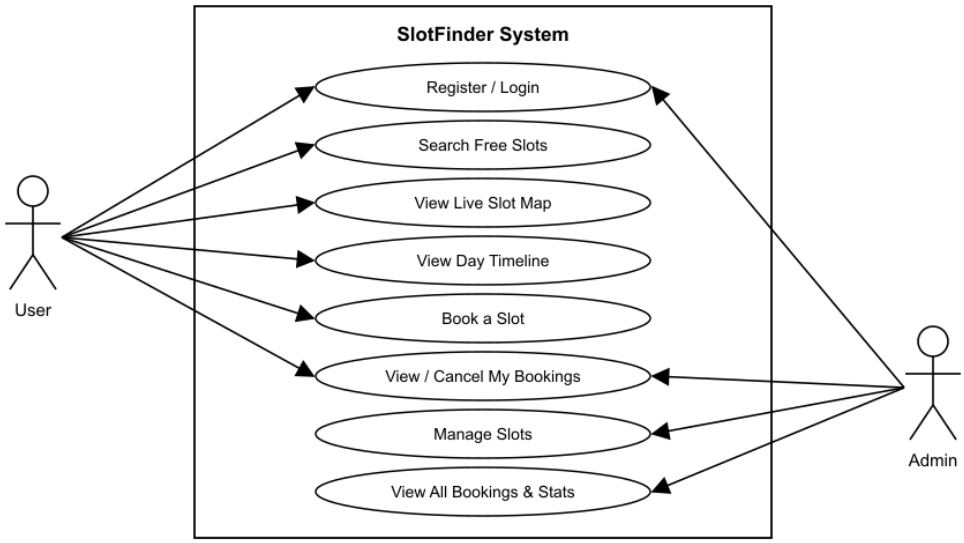
3.3 Website Structure and Navigation Flow

The application is a multi-page website served by Express from the public folder. Public pages include the home page, login and registration. Protected user pages include Find Slot (dashboard), Day View and My Bookings. The Admin Panel is visible only to users with the admin role. A shared navigation bar links these pages, and the JWT stored in the browser decides which pages a visitor may open.



3.4 UML Diagram – Use Case View

The following use case view summarizes the major actors and their responsibilities. The User searches for and books slots and manages their own bookings, while the Admin manages slots and monitors all bookings. Both actors log in through the same authentication process.



3.5 Database Design

The backend connects to MongoDB using Mongoose. The connection string is read from the MONGO_URI environment variable; the default local value points to the database named slotbooking. Data is organized into three collections represented by model files. MongoDB is document-oriented, so a booking document links to its user and slot through ObjectId references.

Collection / Model	Purpose
User	Login identity and role: name, unique email, 10-digit mobile number, bcrypt-hashed password and role (user or admin).
Slot	Parking slot details: unique slot number (e.g. A-01), vehicle type (car, bike, EV, truck), floor, price per hour and active status.
Booking	Reservation record: user and slot references, vehicle number, vehicle type, start time, end time, amount and status (booked or cancelled).

The Booking collection has a compound index on slot, status, start time and end time. This index supports the overlap query used for free-slot identification, so the system can quickly find bookings that collide with a requested window.

MongoDB stores information as documents grouped into collections. Mongoose models provide a defined structure for application records, apply schema validation such as the vehicle number pattern and value ranges, and allow backend code to work with those records through model operations. Passwords are hashed in a pre-save hook before they are written.

Separating collections by purpose makes it easier to maintain user accounts, slot inventory and bookings independently. References between records allow the backend to associate a booking with its user and slot and to populate those details when bookings are listed. The occupancy of a slot is never stored as a flag; it is always computed from the bookings, which keeps the data consistent.

3.6 Tools and Technologies Used

Technology	Use in the project
HTML5 / CSS3	Page structure, responsive layout, slot map, timeline and animations.
JavaScript (ES6)	Client-side logic, live form validation and dynamic page rendering.
Fetch API	HTTP requests from the frontend to backend APIs with the JWT header.

Technology	Use in the project
Node.js and Express.js	Backend runtime, middleware, static file serving and API route handling.
MongoDB and Mongoose	Document database and schema/model layer.
JWT and bcryptjs	Token-based authentication and password hashing.
express-validator	Server-side validation of every request field.
CORS and dotenv	Cross-origin access and environment-variable configuration.
Node.js test runner	Automated API tests on a separate test database.
MongoDB Compass	Viewing and verifying stored collections.
VS Code	Development environment.

3.7 Module Description

User Module

Users can create an account and log in, then search for free slots by vehicle type and time window. Free slots are listed with floor, price per hour and estimated cost, and a slot can be booked by entering the vehicle number and accepting the terms. Users can follow the live slot map, open the day view and manage their bookings under Upcoming, Active now, Completed and Cancelled filters. Protected pages redirect to the login page when no valid session exists.

Free Slot Identification Module

This module, implemented in utils/freeSlots.js, is the core of the project. Two intervals overlap when the existing start time is earlier than the requested end time and the existing end time is later than the requested start time. The module fetches all active slots of the requested vehicle type, finds bookings that are not cancelled and overlap the window, and returns the remaining slots as free. When no slot is free, it walks each slot's future bookings in time order, pushes a candidate start time past every colliding booking, and suggests the earliest time at which any slot is free for the same duration.

Booking Module

When a booking is submitted, the server checks that the slot exists, is active and matches the vehicle type, then re-checks availability for the exact window. If another user

booked the slot in the meantime, the request is rejected with a conflict response. The server also rejects a booking when the same vehicle number already has an overlapping booking. The amount is calculated from the price per hour and the duration. Users can cancel upcoming or active bookings, which immediately makes the slot free again.

Administrator Module

The administrator interface supports adding, editing, deactivating and deleting slots, viewing all bookings with search, cancelling any booking and viewing statistics for total, active and cancelled bookings and revenue. A slot with active or upcoming bookings cannot be deleted and must be deactivated instead. Access to the admin page is restricted on the client, and every admin API is protected by a role check on the server.

CHAPTER 4: IMPLEMENTATION AND RESULTS

4.1 Implementation Details

4.1.1 Front-End Development

The frontend is located in the public directory and consists of HTML pages for the home page, login, registration, Find Slot (dashboard), Day View, My Bookings and the Admin Panel. A shared stylesheet provides the layout, colours, cards, slot map and timeline. Shared scripts handle API calls, the session, the navigation bar, toast messages, date formatting and form validation, while page scripts render the dashboard and admin data.

The live slot map on the Find Slot page refreshes automatically every 30 seconds. The day view draws a 24-hour bar for every slot with booked periods in red and a marker for the current time. The layout adapts to mobile, tablet and desktop screen widths.

The frontend is responsible for rendering pages, collecting user input and providing navigation and status feedback. The Fetch API is used for communication with backend endpoints, and the JWT returned at login is stored in the browser and sent in the Authorization header of each request.

Frontend validation improves usability by showing a message under each field as the user types, for example for the mobile number, password strength or vehicle number format. It does not replace backend validation, because requests can be sent outside the browser interface. Every rule is therefore checked again on the server before any database change is accepted.

4.1.2 Back-End Development

The backend entry point is server.js. It loads environment variables, connects to MongoDB, creates an Express application, applies CORS and JSON body-parsing middleware with a size limit, serves the frontend from the public folder and mounts the route groups for authentication, slots and bookings. A health endpoint reports the server status, unknown API routes return a 404 response, and a central error handler converts invalid JSON, invalid IDs and duplicate keys into clear messages.

Route handlers contain feature-specific operations, models define data structures, and middleware handles cross-cutting tasks such as authentication, role checks and validation. The free-slot logic is kept in a separate utility module so that it can be reused by the search and booking routes. Sensitive values such as the JWT secret and database connection string are stored in the .env file and are not committed to the repository.

JWT-based authentication identifies the logged-in user on every protected request, while password hashing with bcryptjs avoids storing plain-text passwords. Role checks are especially important for administrator actions. Error responses return field-wise messages for validation failures and do not expose credentials or internal stack traces.

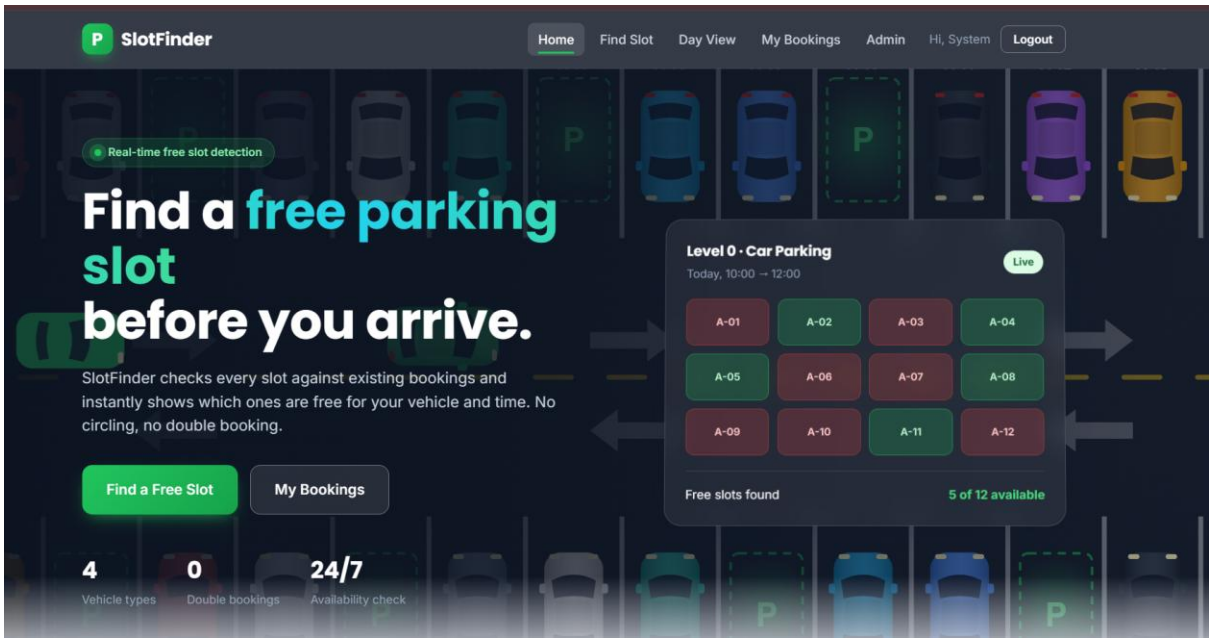
4.1.3 Database Connectivity

The database connection module calls mongoose.connect() using process.env.MONGO_URI. If the variable is missing or the connection fails, the server logs a clear message and exits. When the connection succeeds, the server logs "MongoDB connected: 127.0.0.1/slotbooking" and starts listening on port 5000. The seed script creates the administrator account and sample slots for cars, bikes, EVs and trucks.

The same code works with a local MongoDB server or with MongoDB Atlas, because only the connection string changes. A successful application startup is always accompanied by a successful database connection. Records created through the website, such as user accounts and bookings, can be verified in MongoDB Compass, as shown in the screenshots.

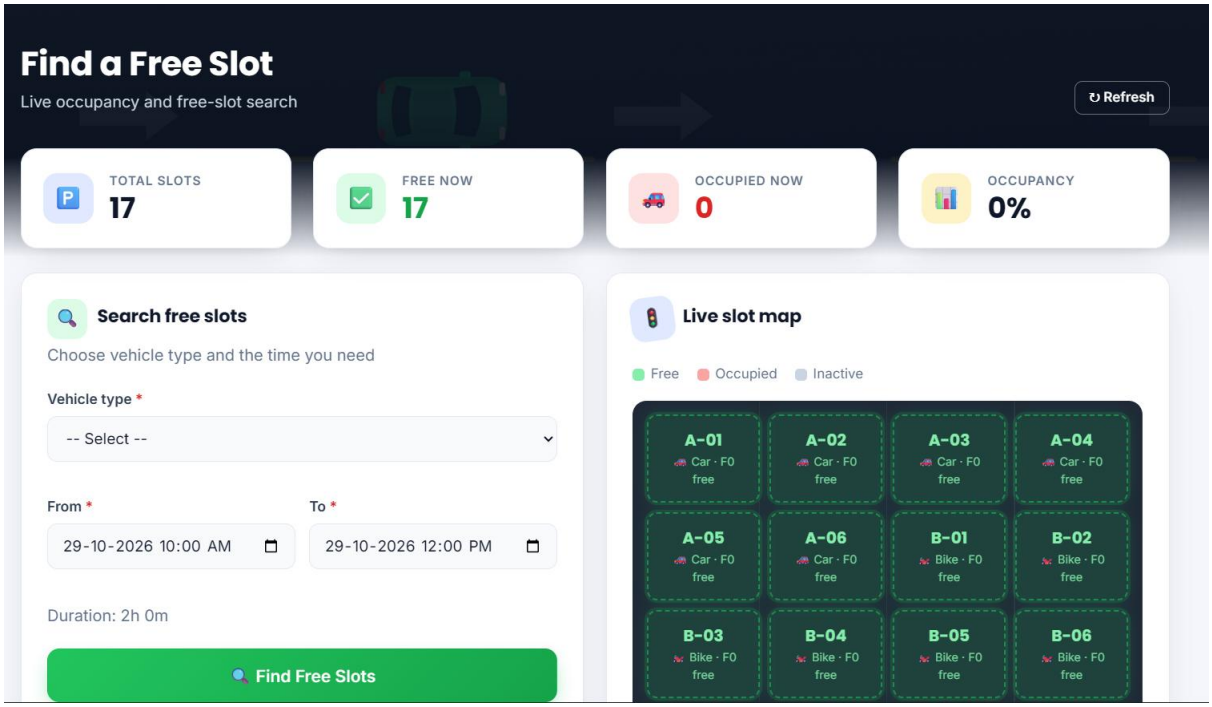
4.2 Screenshots of the Web Application

1. Home Page

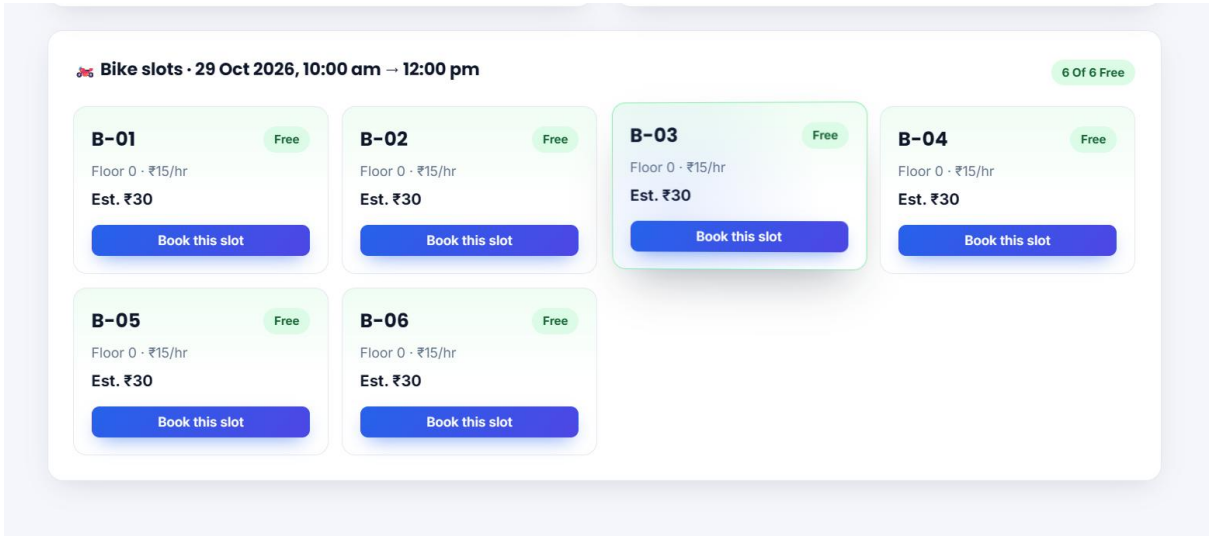


2. User Pages

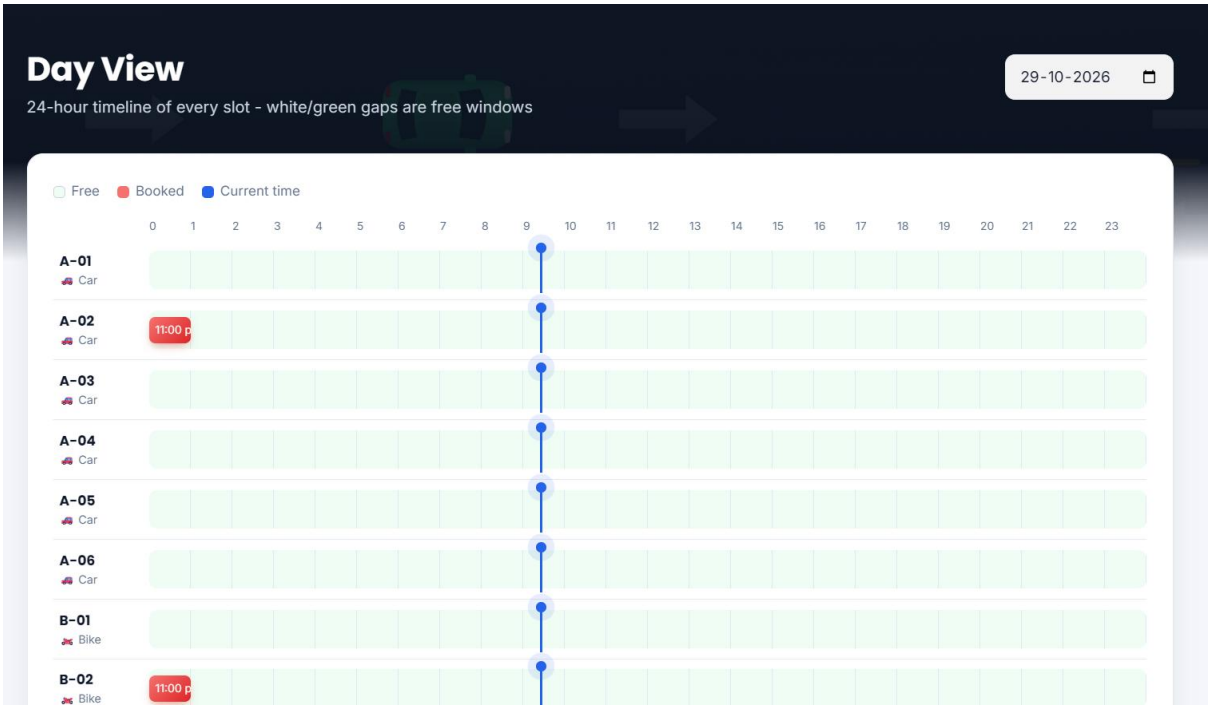
i) Find a Free Slot



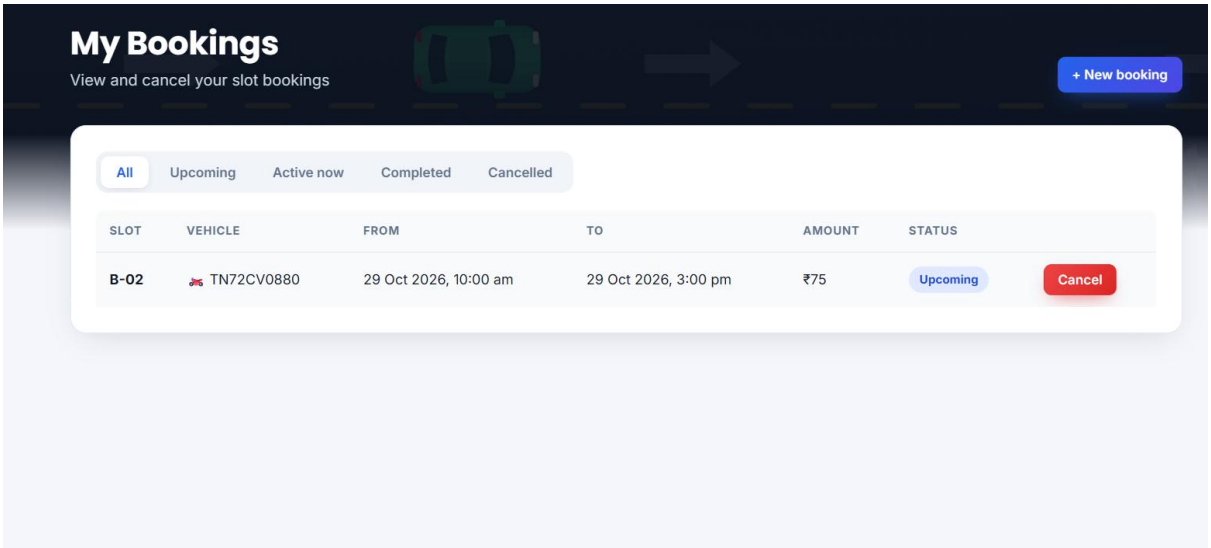
ii) Free Slot Search Results



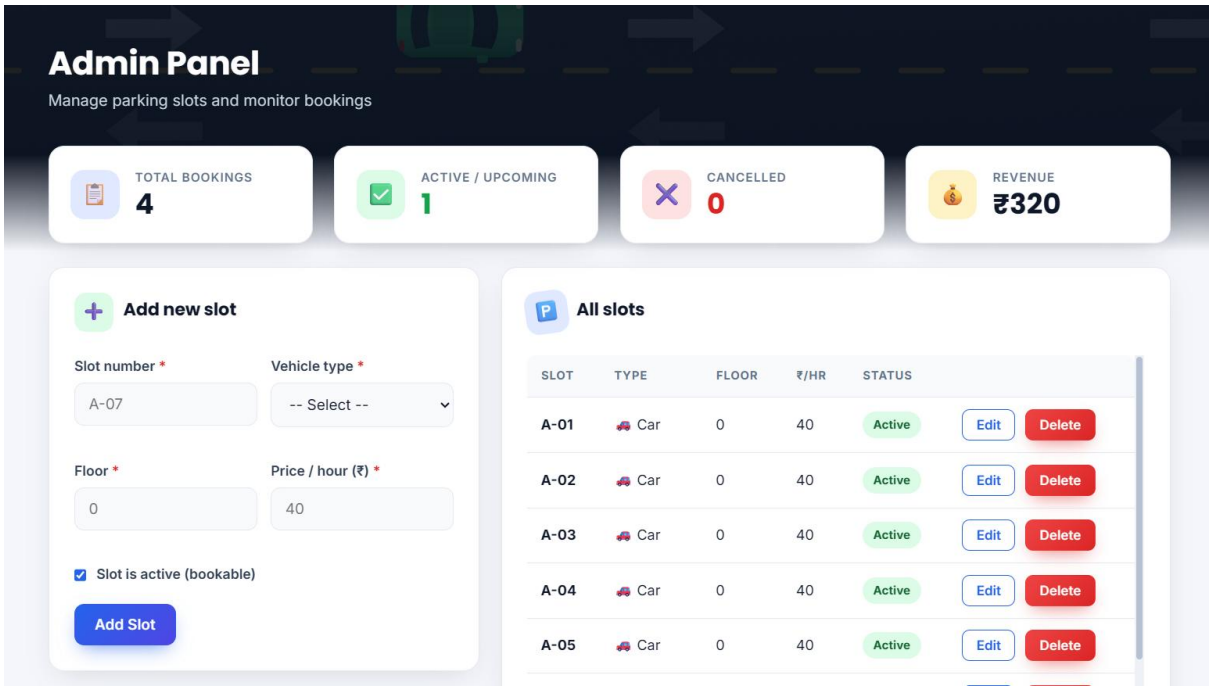
iii) Day View Timeline



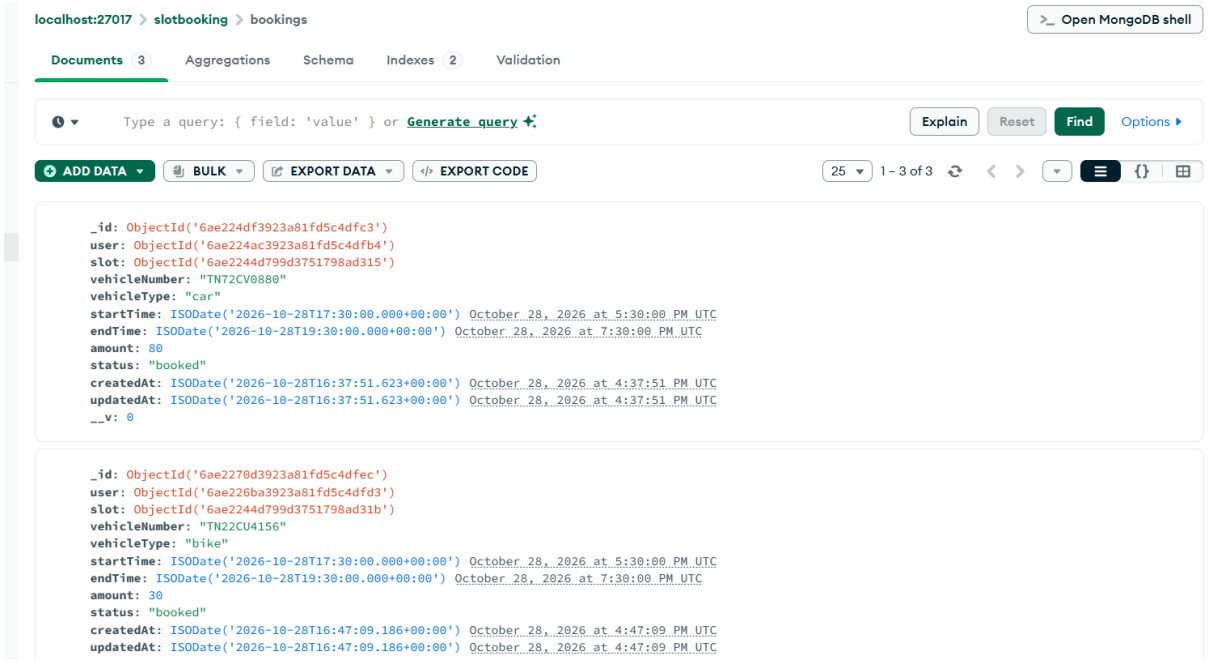
iv) My Bookings



3. Admin Page



4. Database



4.3 Sample Inputs and Outputs

The following examples show inputs given to the application and the outputs it returns, as observed in the running project and its automated tests.

Operation	Sample Input	Observed Output
Registration	Name, email, 10-digit mobile number and strong password	Account is created and a login token is returned.
Invalid registration	Mobile number with fewer than 10 digits	Request is rejected with the message "Enter a valid 10-digit mobile number".
Free slot search	Vehicle type Bike, 29-10-2026 10:00 AM to 12:00 PM	6 of 6 bike slots listed as free, each with estimated cost ₹30.
Book a slot	Slot B-02, vehicle TN72CV0880, 10:00 AM to 3:00 PM	Booking is created with amount ₹75 and appears under Upcoming.
Double booking	Same slot for an overlapping time	Request is rejected with a conflict message asking the user to search again.
Invalid vehicle number	Vehicle number not in Indian format	Rejected with "Vehicle number must look like TN01AB1234".
All slots full	Search when every slot of the type is booked	No free slots; the earliest next free time is suggested.
Cancel booking	Cancel an upcoming booking	Status changes to cancelled and the slot becomes free again.
Add slot (admin)	Slot A-07, Car, floor 0, ₹40 per hour	Slot is created and listed under All slots.

4.4 Testing and Results

Testing verified the behaviour of individual functions and the integration between the frontend, the API and the database. The project contains an automated test suite (tests/api.test.js) that runs against a separate slotbooking_test database, which is deleted afterwards so that real data is not touched. The table below lists the test cases and the results observed when the suite was executed with npm test.

Test Case	Procedure	Expected Result	Observed Result
API health	Start the server and open /api/health	JSON status response is returned	Pass

Test Case	Procedure	Expected Result	Observed Result
Database connection	Start MongoDB and the server	Connection message appears in server log	Pass
Form validation	Submit invalid or missing registration fields	Invalid input is rejected with field errors	Pass
Password security	Register and inspect the stored user	Password is stored as a hash	Pass
Role protection	Call admin and protected routes without the correct role or token	Access is blocked with 401 / 403	Pass
Free slot search	Search a time window before and after booking	Booked slot disappears only for overlapping windows	Pass
Double booking	Book the same slot for an overlapping window	Second booking is rejected	Pass
Vehicle clash	Book one vehicle into two slots at once	Second booking is rejected	Pass
Next free suggestion	Fill all slots and search again	Earliest free time is suggested	Pass
Cancellation	Cancel a booking and search again	Slot is free again	Pass
Admin slot management	Add, edit, deactivate and delete slots	Changes are saved; slot with upcoming bookings is not deleted	Pass
Error handling	Send invalid IDs and unknown routes	Clear 400 / 404 responses	Pass

Unit and integration testing: The free-slot functions and API routes were tested together through 24 automated test cases covering registration, login, validation, search, booking, double-booking prevention, the next-free suggestion, cancellation and administrator operations. Functional testing: end-to-end workflows such as registration, searching, booking and cancelling were also carried out in the browser, as shown in the screenshots. Security testing: authorization was verified on the backend as well as in the frontend page guards, and inputs were validated on both sides.

Result statement: The application was evaluated with the automated test suite on 29 October 2026. All 24 test cases passed and none failed, with a total run time of about 40 seconds.

Testing covered both expected user actions and invalid or unauthorized actions. Examples include submitting incomplete registration details, calling a protected route without a valid session, booking a slot for the wrong vehicle type, booking the same slot twice and checking that a normal user cannot reach administrator routes. These tests identify problems that are not visible from screenshots alone.

CHAPTER 5: CONCLUSION AND FUTURE ENHANCEMENT

5.1 Summary of the Work

SlotFinder identifies free parking slots for a chosen vehicle type and time window in one full-stack application. It organizes work into user and administrator modules and provides free-slot search with estimated cost, booking with double-booking prevention, next-free suggestions, a live slot map, a 24-hour day view, booking management and administrative slot management with statistics. The frontend is implemented with HTML5, CSS3 and JavaScript, the backend with Node.js and Express, and persistent data is managed in MongoDB through Mongoose. Supporting libraries provide authentication, password hashing and server-side validation.

5.2 Conclusion

The project demonstrates how multiple Web Technology concepts can be integrated into a practical vehicle booking system. It illustrates page design, client-side validation, client-server API communication, database-backed application logic, authentication, role-based access and the use of an interval-overlap algorithm to solve a real problem. Separating user and administrator responsibilities improves the organization of the workflows, and the automated tests confirm that the main functions behave as expected.

The project brings together core web development concepts: responsive user interfaces, HTTP API communication, backend request processing, database persistence, authentication and validation. It also shows how computing availability from stored bookings, instead of storing a simple occupied flag, keeps the data consistent and makes double booking impossible.

The quality of the final demonstration depends on connecting the complete workflow and showing evidence from the running application. The genuine screenshots and recorded test results included in this report make it reliable and easier to defend during evaluation.

5.3 Future Enhancements

- **Online payment gateway :** integration (for example Razorpay or Stripe) with payment confirmation and refunds on cancellation.
- **IoT sensor integration :** sensors or cameras at each slot to confirm physical occupancy and detect vehicles that park without booking.

- **QR code entry and exit :** a QR code on each booking, scanned at the gate to start and end the parking session.
- **Notifications :** email or SMS reminders before a booking starts and alerts when the booked time is about to end.
- **Multiple parking locations :** support for several parking sites with a map view and distance-based search.
- **Dynamic pricing :** different prices for peak hours, weekends and long-duration bookings.
- **Mobile application :** an installable progressive web app or native mobile client.
- **Analytics and reports :** occupancy trends, peak-hour charts and downloadable reports for the administrator.
- **Security hardening :** rate limiting, refresh tokens, a password-reset flow and deployment with HTTPS.

REFERENCES

- OpenJS Foundation, "Node.js Documentation," https://nodejs.org/en/docs
- Express.js, "Express – Node.js web application framework," https://expressjs.com
- MongoDB Inc., "MongoDB Manual," https://www.mongodb.com/docs/manual/
- Mongoose, "Mongoose ODM Documentation," https://mongoosejs.com/docs/
- M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, 2015. https://datatracker.ietf.org/doc/html/rfc7519
- jsonwebtoken, "jsonwebtoken – npm," https://www.npmjs.com/package/jsonwebtoken
- bcrypt.js, "bcryptjs – npm," https://www.npmjs.com/package/bcryptjs
- express-validator, "express-validator Documentation," https://express-validator.github.io/docs/
- Mozilla Developer Network, "Fetch API," https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- Mozilla Developer Network, "MDN Web Docs," https://developer.mozilla.org
- OpenJS Foundation, "Node.js Test Runner," https://nodejs.org/api/test.html
- MongoDB Inc., "MongoDB Compass," https://www.mongodb.com/products/tools/compass
- W3Schools, "Web Development Tutorials," https://www.w3schools.com
- R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, University of California, Irvine, 2000.
- OWASP Foundation, "OWASP Top Ten Web Application Security Risks," https://owasp.org/www-project-top-ten/